
Weike Liu

| RL, MicroDuck, MuJoCo

TABLE OF CONTENTS

- 1 Install MuJoCo
- 2 First Hello World
- 3 Look at some real stuff
- 4 What is MuJoCo?
- 5 Building a robotic arm
 - 5.1 Step 1: Define the robot model (MJCF/XML file)
 - 5.2 Step 2: Parameter configuration file (YAML)
 - 5.3 Step 3: Main Python program (complete development workflow)
 - 5.3.1 Import dependencies and initialization
 - 5.3.2 Load YAML configuration (`load\_config` function)
 - 5.3.3 Load MuJoCo model
 - 5.3.4 Enter simulation main loop (with passive viewer)
 - 5.3.5 Main loop internals (`while viewer.is\_running()`)
 - 5.3.6 Exit loop and finish
 - 5.3.7 Additional notes
 - 5.4 Step 4: How to run and verify the results
- 6 Back to the little duck
 - 6.1 MicroDuck main repository
 - 6.1.1 Overall structure
 - 6.1.2 MicroDuck robot control core: robotd
 - 6.1.2.1 robotd's execution flow
 - 6.1.2.2 Part 1: robotd main flow
 - 6.1.2.2.1 Block 1: Parameters and configuration
 - 6.1.2.2.2 Block 2: robotd init is a special path
 - 6.1.2.2.3 Block 3: Preempt robotd's Unix socket
 - 6.1.2.2.4 Block 4: Create RobotState
 - 6.1.2.2.5 Block 5: Create Intents

-
- 6.1.2.2.6 Block 6: Create poweroff callback
 - 6.1.2.2.7 Block 7: Start the control thread
 - 6.1.2.2.8 Block 8: Main thread starts IPC server
 - 6.1.2.2.9 How does the IPC server itself work?
 - 6.1.2.2.10 What does the IPC handler do?
 - 6.1.2.2.11 When does the main thread exit?
 - 6.1.2.3 Second main line: 50 Hz Control Loop
 - 6.1.2.3.1 Control Loop initialization phase
 - 6.1.2.3.2 Block 1: Determine current policy
 - 6.1.2.3.3 Block 2: Create Safety
 - 6.1.2.3.4 Block 3: Read startup pose
 - 6.1.2.3.5 Block 4: Create Controller
 - 6.1.2.3.6 Block 5: Establish 50 Hz ticker
 - 6.1.2.3.7 Enter while
 - 6.1.2.3.8 Tick Block 1: Wait for cycle
 - 6.1.2.3.9 Tick Block 2: Read sensors
 - 6.1.2.3.10 Tick Block 3: Coast
 - 6.1.2.3.11 Tick Block 4: Update IMU / odometry / fall status
 - 6.1.2.3.12 Tick Block 5: Read Intent Snapshot
 - 6.1.2.3.13 Tick Block 6: Safety Gate
 - 6.1.2.3.14 Tick Block 7: Handle one-time power request
 - 6.1.2.3.15 Tick Block 8: Handle Skill / Sit / Ground Pick
 - 6.1.2.3.16 Tick Block 9: Handle sound
 - 6.1.2.3.17 Tick Block 10: Handle Mode Switch / Policy Reload
 - 6.1.2.3.18 Tick Block 11: Shutdown / Battery Shutdown
 - 6.1.2.3.19 Tick Block 12: Limp-Fall
 - 6.1.2.3.20 Tick Block 13: Decide whether to enter driving
 - 6.1.2.3.21 Tick Block 14: Handle driving start / stop
 - 6.1.2.3.22 Tick Block 15: Battery voltage compensation

-
- 6.1.2.3.23 Tick Block 16: Generate Joint Targets
 - 6.1.2.3.24 Tick Block 17: Controller.step()
 - 6.1.2.3.25 Tick Block 18: Theremin / Chorale / Mouth
 - 6.1.2.3.26 Tick Block 19: Finally enter Safety
 - 6.1.2.3.27 Tick Block 20: Publish RobotState
 - 6.1.2.3.28 Tick Block 21: Record loop health

6.1.2.4 Some key design points of robotd

6.2 robotd's ONNX model declaration, instantiation, and invocation

- 6.2.1 Define: define an ONNX model object
- 6.2.2 Create: load the `.onnx` file into a `Session`
- 6.2.3 `robotd` creates it
- 6.2.4 Execute: call ONNX
- 6.2.5 How does `robotd` call it?
- 6.2.6 Summary of the three actions

6.3 From ONNX file to program output: the complete flow

- 6.3.1 First understand: what an ONNX file itself is
- 6.3.2 The complete process (four steps)

7 MicroDuck Training

8 Browser simulation tool (visual understanding of architecture operation)

9 Official documentation and feature description (supplementary architecture design philosophy)

10 RL Algorithms

1 INSTALL MUJoCo

On Windows 11, MuJoCo has long been a pure Python package. There is no need to apply for a license or configure complex environment variables.

Open Command Prompt (cmd) or PowerShell.

Run the installation command: `pip install mujoco`

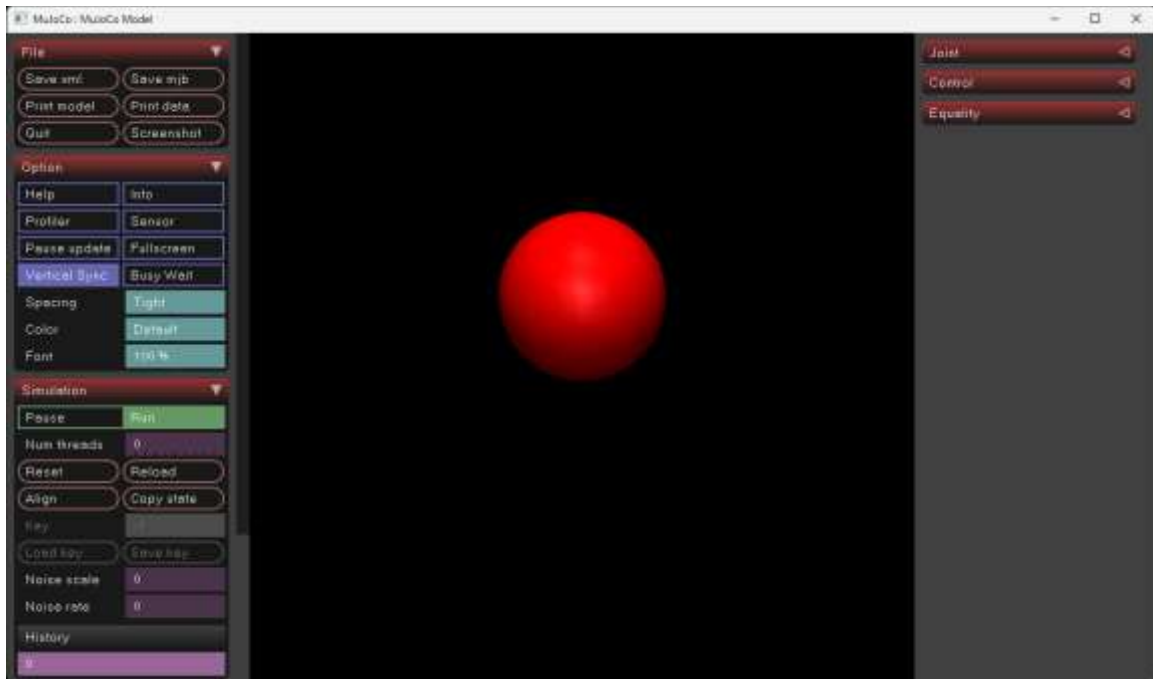
This command will automatically install the core library and rendering dependencies.

After installation, you can verify with: `python -c "import mujoco; print(mujoco.__version__)"`

2 FIRST HELLO WORLD

[HelloWorld.py](#)

Run: `python HelloWorld.py`, and it immediately appears:



3 LOOK AT SOME REAL STUFF

Install yaml, torch, etc.

Pip install pyyaml

Pip install torch

```
git clone https://github.com/limxdynamics/humanoid-description.git
```

Enter the directory, then:

```
python -m mujoco.viewer --mjcf=HU_D04_description/xml/HU_D04_01.xml
```

Additionally, there is the Unitree robot: `git clone https://github.com/yueruyin/robot-unitree-g1.git`

4 WHAT IS MUJoCo?

The core of MuJoCo is a physics engine library written in C. Python is just one of the most popular and convenient ways to call it.

Specifically, you have the following 4 usage approaches:

- ◆ Python interface (most commonly used): Installed via `pip install mujoco`. This is the officially maintained Python binding that wraps the C library into a Python module. This is the method you are currently using to run scripts and view models.
- ◆ C/C++ native call: If you are doing low-level robot development or embedded systems, you can directly download MuJoCo's C library (.dll/.so/.dylib) and call its physics calculation functions directly in C/C++ code.
- ◆ Command-line tools: After installing the Python package, command-line tools are automatically included. For example, the `python -m mujoco.viewer --mjcf=xxx.xml` you used before essentially calls the built-in visualization program.
- ◆ Bindings for other languages: The community also provides Julia (MuJoCo.jl), Rust (mujoco-rs), and even MATLAB interfaces, covering most mainstream programming languages.

Summary: Python is the preferred choice for getting started and doing reinforcement learning (RL), but MuJoCo itself is very "general-purpose" and not tied to any language.

1. Core Positioning: Library vs. Platform

MuJoCo is a "library": It's like OpenCV (computer vision library) or FFmpeg (audio/video library). It is a set of low-level computation tools with no "main interface". Your code is the "commander", calling its API to compute physics, contact forces, and joint angles, then you draw the results with other tools. It works entirely around code.

Isaac Sim is a "platform/application": It's like Unity or Unreal Engine. It is a massive graphical integrated environment. Most of your time is spent dragging and dropping models, building scenes, setting lights in the GUI, and clicking buttons to run simulations. Code (Python) is just a "plugin" to assist GUI operations or write complex logic.

2. The "trap" of the GUI (something many people misunderstand)

You might ask: "But doesn't `python -m mujoco.viewer` also pop up a window (GUI)?"

Note the distinction: The window that comes with MuJoCo is essentially a "debug monitor", not a "development platform".

Isaac Sim's GUI: You work inside it (building scenes, modifying physics parameters, viewing render effects).

MuJoCo's viewer: You just use it as a display (to see how the robot is actually running, whether its legs are shaking, whether it has fallen). Once you close this viewer, your Python script can still compute physics data in the background (such as logging or training AI), while Isaac Sim ends the entire simulation environment once you close the GUI.

5 BUILDING A ROBOTIC ARM

5.1 STEP 1: DEFINE THE ROBOT MODEL (MJCF/XML FILE)

MuJoCo uses the MJCF (MuJoCo XML) format to describe robots. This file defines the geometry, joints, mass, colors, etc. of the robotic arm. Create a [two link arm.xml](#) file.

5.2 STEP 2: PARAMETER CONFIGURATION FILE (YAML)

YAML is used here to store "control algorithm parameters", avoiding hardcoding them in Python code: [config.yaml](#).

5.3 STEP 3: MAIN PYTHON PROGRAM (COMPLETE DEVELOPMENT WORKFLOW)

[run_arm.py](#), this is the core code. It integrates model loading, YAML reading, control loop, and visualization all together.

The program has excellent comments and is very easy to read.

This Python program (`run_arm.py`) implements a two-link robotic arm trajectory tracking simulation based on the MuJoCo physics engine. It sets joint target trajectories through a YAML configuration file and uses the built-in `position` actuator in the XML model (with built-in PD control) to directly send desired angles, thereby driving the robotic arm's motion. Below is a step-by-step explanation of its workflow in execution order:

5.3.1 Import dependencies and initialization

Import `time`, `numpy`, `yaml`, `mujoco`, and its `viewer` submodule. These libraries are used for time control, numerical computation, configuration file parsing, physics simulation, and visualization, respectively.

5.3.2 Load YAML configuration (`load_config` function)

Attempt to open the `config.yaml` file and parse it into a dictionary.

- If the file does not exist, use default parameters (containing only trajectory amplitude, frequency, and phase difference) and display a warning.

- The configuration content is used to generate the target angle trajectories for the two joints.

5.3.3 Load MuJoCo model

Read the XML model file via ``mujoco.MjModel.from_xml_path("two_link_arm.xml")``.

If the file fails to load, the program prints an error and exits.

Then create an ``MjData`` object to store simulation state (position, velocity, control variables, etc.).

5.3.4 Enter simulation main loop (with passive viewer)

Use ``mujoco.viewer.launch_passive()`` to start a passive (non-blocking) visualization window, passing in the model and data.

Record the start time ``start_time`` for calculating the elapsed simulation time ``elapsed``.

5.3.5 Main loop internals (``while viewer.is_running()``)

Each iteration executes the following steps:

a. Calculate desired joint angles

- Based on parameters such as ``amplitude1`` and ``freq1`` in the configuration, use sine functions to generate the target angles for the two joints:

- ``target1 = A1 * sin(2π * f1 * t)``

- ``target2 = A2 * sin(2π * f2 * t + φ2)``

- Here ``t`` is the simulation runtime (physical time, not CPU time).

b. Directly send target angles

- Assign the calculated ``target1`` and ``target2`` to ``data.ctrl[0]`` and ``data.ctrl[1]``.

- Key point: The joint actuator type in the XML model is ``<position>``, which has ``kp`` (proportional gain) and ``kv`` (velocity gain) defined internally. Therefore, MuJoCo automatically converts angle errors into torques, ****without needing to manually compute the PD control law in Python****.

c. Step the physics engine

- Call ``mujoco.mj_step(model, data)`` to advance the simulation by one step, updating all states including position, velocity, and torque.

d. Sync visualization

-
- Call `viewer.sync()` to render the current data to the display window.

e. Control simulation speed

- Use `time.sleep(model.opt.timestep)` to make the program sleep for one physics timestep (typically a few milliseconds), ensuring the simulation runs at roughly real-time speed (real-time simulation) and preventing the loop from running too fast.

5.3.6 Exit loop and finish

- When the user closes the viewer window or presses the stop key, `viewer.is_running()` becomes `False`, and the loop terminates.
- Print `"Simulation ended"` and the program exits.

5.3.7 Additional notes

- The entire process is a closed loop of `***Configuration → Model loading → Real-time trajectory generation → Direct position control → Physics simulation → Rendering***`.
- This design places low-level control (PD parameters) in the XML, while the upper-level Python only handles trajectory planning, achieving separation of control logic from physics simulation for easier debugging and parameter adjustment.
- The real-time performance of the simulation depends on `time.sleep`, but the actual physics timestep is determined by `model.opt.timestep` (usually fixed), and `sleep` is only for throttling. If the computer performance is insufficient, the simulation may slow down, but it will not affect physics accuracy.

5.4 STEP 4: HOW TO RUN AND VERIFY THE RESULTS

Run the script: `python run_arm.py`

You will see: a green + blue robotic arm, with the base rotating (joint 1), while the upper arm swings up and down (joint 2), and the orange ball at the end traces an elegant "8" figure trajectory. You can drag to rotate the view with the mouse, right-click to pan, and scroll to zoom.

Through this example, understand MuJoCo development's "four-step method"

Stage	File/Code	Role
① Model definition	*.xml (MJCF)	This is the robot's "physical blueprint", defining geometry, mass, joint types (hinge/ball/free), friction, colors, etc. The essence of

		MuJoCo lies in tuning the XML parameters.
② Parameter configuration	*.yaml	This is the "tuning sheet for control algorithm parameters". It moves frequency, amplitude, and gains out so you can tweak them repeatedly during experiments without frequently changing the Python code.
③ Core logic	Python script	Responsible for loading (model + YAML), computing (target trajectories), executing (assigning data.ctrl), and stepping (mj_step). This is where you write your algorithms.
④ Visualization loop	viewer.launch_passive	The passive loop keeps you in control — compute control first, then step the physics, and finally refresh the view. This is more suitable for algorithm development than launch (fully automatic).

Advanced tips

Adjust parameters: Modify amplitude or freq in config.yaml, save and re-run to observe changes in the robotic arm's swing, without changing Python.

Export data: Add a line `print(data.body('tip').xpos)` in the loop to print the end-effector's 3D position in real-time for trajectory analysis.

Change to joint space following: If you want it to maintain a fixed posture, just hardcode `target1 = 0.5`.

This framework covers the basic structure of virtually all MuJoCo simulation projects. You can replace `two_link_arm.xml` with the Unitree G1 model and replace the sine wave with a reinforcement learning policy to upgrade to an advanced project.

6 BACK TO THE LITTLE DUCK

If you are not familiar with ONNX yet, you can first read 6.3.

MicroDuck has these official open-source repositories:

- Official main repository, the core repository of MicroDuck, containing the SDK and robot body software.

Address: <https://github.com/pollen-robotics/microduck/tree/main>

License: Apache-2.0, which means you are free to use, modify, copy, and distribute the code. Including:

Commercial use: You can use the code in commercial software or internal projects.

Closed-source distribution: You can integrate the code into your own products without open-sourcing your own source code.

Relicensing: You can distribute the entire work (including Apache-2.0 code) under other licenses.

Obligations you must comply with:

Retain notices: When distributing code, you must retain all copyright notices, license declarations, and attribution information in the NOTICE file from the original code.

Mark modifications: If you modify files, you need to clearly mark where the changes were made.

Include license: When redistributing, you need to include a copy of the Apache-2.0 license.

- Official reinforcement learning repository, this repository focuses on reinforcement learning and sim-to-real deployment tools. Address: https://github.com/pollen-robotics/microduck_rl
- Official related tools

MicroDuck Studio: a local web development studio for debugging, visualizing, and orchestrating MicroDuck workflows. Address: <https://github.com/microai-lab/microduck-studio>

- Additionally, there is a third-party replica project, a community-initiated third-party replication project aimed at reconstructing assembly drawings, CAD assemblies, etc. from the officially published simulation models and source code. Address: <https://github.com/fanhao375/microduck-replica>

Note that MicroDuck's software is open source (Apache-2.0), but the hardware itself is not open source.

6.1 MICRODUCK MAIN REPOSITORY

Address: <https://github.com/pollen-robotics/microduck/tree/main>. This repository contains the software running on MicroDuck's RK3566 processor. MicroDuck sometimes refers to it as "runtime" software.

Microduck is not a "single program", but rather a Rust-based multi-process robot operating system/runtime running on the robot's RK3566 Linux board.

The core is robotd, which handles the actual robot closed-loop control; other daemons ¹ around it handle configuration, Bluetooth, gamepad, camera/remote communication, ToF, and software updates. All control services communicate via Unix Socket + JSON-RPC 2.0/NDJSON.

6.1.1 Overall structure

The repository itself is a Rust workspace. Currently, the Cargo workspace ² has 19 members, including daemon, control algorithm library, CLI, IPC protocol, tools, etc.

¹In Linux, a daemon is a special process that runs in the background, silently providing services without any interactive interface. You can think of it as a "backstage worker" or "system attendant" in your computer: you don't need to talk to it directly (no GUI or command-line interaction); it is always on standby (continuously running in memory); when there is a request, it works (responding to client calls); when there is no request, it idles (using minimal CPU resources). One point that is easy to confuse:

daemon: runs in the background, no interaction, provides system services.

While "service" is more of a "functional-level" term. The daemon is the concrete process that implements a service; the two are often used interchangeably. For example, the "SSH service" is provided by the sshd daemon.

How to create your own daemon (briefly): if you write it in C, the standard steps are:

fork() creates a child process and the parent exits; the child calls setsid() to create a new session; close the file descriptors (stdin/stdout/stderr); change the working directory to /; set the file permission mask umask(0), etc.

However, nowadays it is recommended to write a systemd service unit file directly and let systemd manage it for you, avoiding these low-level details.

²Cargo Workspace is a proper noun specific to the Cargo tool in the Rust ecosystem. It is a core feature designed by the official Rust team for managing multiple interrelated packages, with no fully equivalent native concept in other programming languages' build systems. Core definition:

A Cargo Workspace is a collection of interrelated Rust packages (crates) developed collaboratively. All member packages share the same Cargo.lock dependency lock file and the same target compilation output directory, fundamentally avoiding dependency version fragmentation and repeated compilation issues in multi-package development.

Core features:

Unified multi-package management: you can split a large project into multiple independent crates with single responsibilities, all maintained collaboratively in the same workspace. A typical case is the official Rust rustc compiler repository — the entire project is a giant Cargo Workspace managing nearly 50 interdependent sub-crates.

Shared configuration and dependencies: [workspace.dependencies] in the root Cargo.toml can define globally shared dependencies, and all sub-crates can directly inherit these dependency versions without repeatedly writing configuration, greatly reducing the probability of dependency version conflicts.

It has these "entry points":

- Linux/systemd service entry point

The programs that truly run persistently after robot startup are mainly:

Entry	Type	Core role
robotd	daemon	Robot motion control core
configd	daemon	WiFi, robot identity, pairing, system configuration
updaterd	daemon	Software/model updates, verification, rollback
btd	daemon	BLE API entry
padd	daemon	Gamepad entry
mediad	daemon	Camera/microphone/WebRTC/remote API
tofd	daemon	ToF depth sensor

These 7 are the most core service divisions of the entire runtime. The architecture documentation also explicitly states that systemd is their supervisor, responsible for lifecycle, crash restart, startup order, and watchdog.

- User/developer CLI entry point

It also has two very important CLIs (Command Line Interfaces): robotctl, duckctl.

✧ Robotctl: This is the management CLI executed directly on the robot's Linux board.

It can perform operations such as:

robotctl version

robotctl health

robotctl monitor

robotctl configure

robotctl policy ...

Flexible build control: you can run cargo build at the root to compile all member packages at once, or use the -p package-name parameter to build, test, or publish a specific sub-crate individually, balancing overall build efficiency and single-package iteration flexibility.

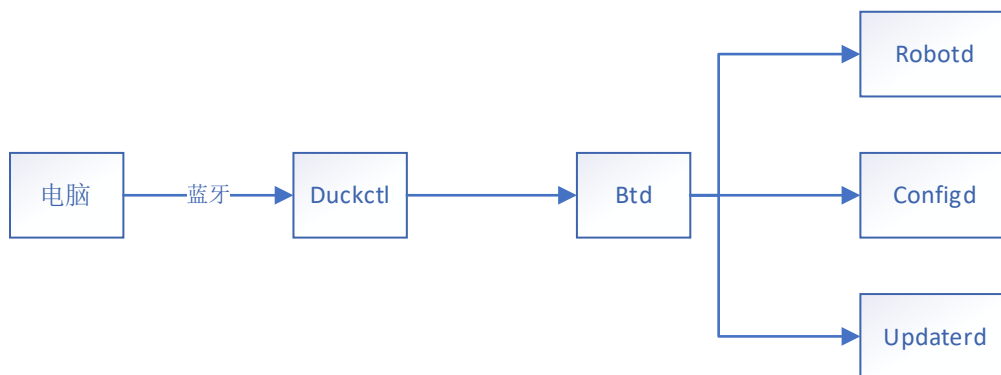
Two manifest modes: the virtual manifest mode (the root Cargo.toml contains only [workspace] configuration and is not itself an independent crate) and the mode where the root is a member package, adapting to different project organization needs.

```
robotctl robot ...  
robotctl pad ...  
robotctl update ...  
robotctl wifi ...  
robotctl system ...  
robotctl voice ...
```

The official cheat sheet defines robotctl as the main control/diagnostic entry point on the robot.

- ✧ Duckctl: this is the CLI on the computer outside the robot. It finds Microduck via Bluetooth, then sends the same JSON-RPC requests as the robot's internal ones.

Therefore:



The code comments explicitly state that duckctl is a "stand-in for the phone app", used only for development and not deployed to the robot.

6.1.2 MicroDuck robot control core: robotd

It is the real-time control daemon that truly makes the duck move. robotd executes a 50Hz control loop, drives 15 servos, and runs neural network policies. robotd is the only daemon that can control motors, and the reason will be explained later. MicroDuck is written in Rust, so many concepts involve Rust-specific concepts, such as slot, tokio, etc., as described below.

6.1.2.1 robotd's execution flow

MicroDuck's robotd is written in Rust, and its entry point is:

```
#[tokio::main]
```

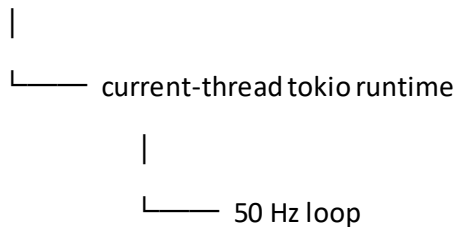
```
async fn main() -> ExitCode
```

After program startup, main()

-
1. reads robotd.toml
 2. creates /run/robotd.sock
 3. creates RobotState + Intents
 4. starts control thread
 5. the main tokio runtime runs the IPC server

A very noteworthy point: the Control loop started in step 4 is a thread, not sharing a runtime with the step 5 IPC. The code specifically creates:

OS thread: control



The reason is that Dynamixel serial I/O is blocking. If control loop and IPC are placed in a normal tokio runtime:

IPC request



tokio worker



blocking serial read



The entire worker gets occupied, and other tasks in the IPC loop may affect the control cycle.

Therefore, consider main.rs as a "main flow + control loop", focusing only on two main lines:

1. robotd main flow: How the process establishes services, creates shared state, starts control threads, and provides IPC after startup.
2. 50 Hz control loop: The execution order of each tick in the control thread.

Below is based on the current main branch code. robotd/src/main.rs currently has about 7600 lines, but the core areas that truly determine the runtime main flow are concentrated in main(), spawn_control_thread(), and control_loop().

6.1.2.2 Part 1: robotd main flow

main() is the true process entry point. The code starts from:

```
#[tokio::main]
```

```
async fn main() -> ExitCode
```

What is actually done here is a series of initialization and assembly, not controlling the robot.
In code order:

```
main()
```

- └─ 1. Parse command line arguments
- └─ 2. Initialize logging
- └─ 3. Load robotd.toml
- └─ 4. Handle --port / --no-policy
- └─ 5. If it's `robotd init`, follow the separate init flow
- └─ 6. claim Unix socket
- └─ 7. Create RobotState
- └─ 8. Create Intents
- └─ 9. Create poweroff callback
- └─ 10. Create control thread
- └─ 11. Start IPC server
- └─ 12. Wait for server / SIGTERM

So main() essentially assembles the runtime environment for robotd, then starts two main execution entities: control thread + IPC server.

6.1.2.2.1 Block 1: Parameters and configuration

Params is static configuration. For example: bus port, control period, policy, safety, deadman, gain, limp fall, audio, mode, etc. The things that change continuously during runtime are not placed in Params, but rather in: RobotState, Intents, Controller - these three different layers.

6.1.2.2.2 Block 2: robotd init is a special path

robotd init is a CLI command. Its execution follows a separate path. After initialization completes, the robot is in "establishing the robot's known posture" state, rather than the normal continuous control service.

6.1.2.2.3 Block 3: Preempt robotd's Unix socket

Only then is it allowed to start the subsequent control thread. The code explicitly requires owning the endpoint before starting threads that may access motors.

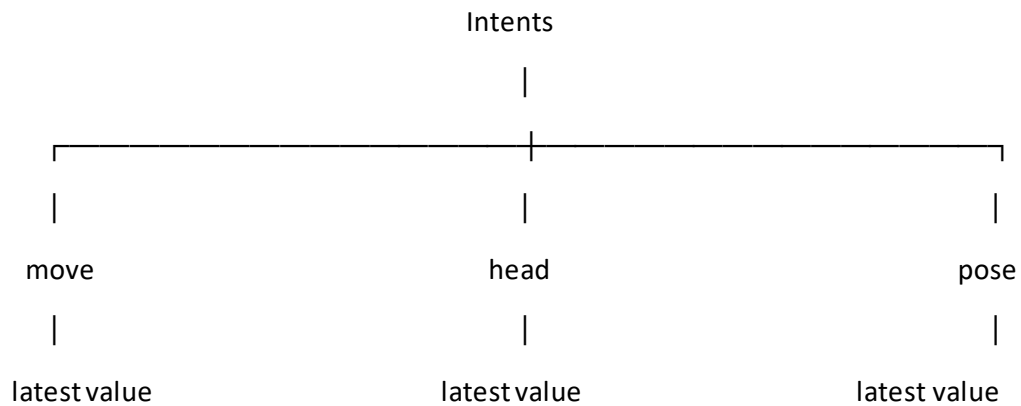
6.1.2.2.4 Block 4: Create RobotState

Next: `let state = Arc::new(RobotState::new(...));` This RobotState is very important. It is not a "Mutex of all the robot's current states". It is primarily: the state publishing channel from control loop → IPC.

6.1.2.2.5 Block 5: Create Intents

Next: `let intents = Arc::new(Intent::new());`

This object is the command exchange area between IPC and the control loop. Note that it is not an ordinary command queue. It is more like:



For example, the client continuously sends:

`robot.move(vx=0.1)`

`robot.move(vx=0.2)`

`robot.move(vx=0.3)`

The control loop does not consume them one by one:

0.1

0.2

0.3

Instead, at each tick:

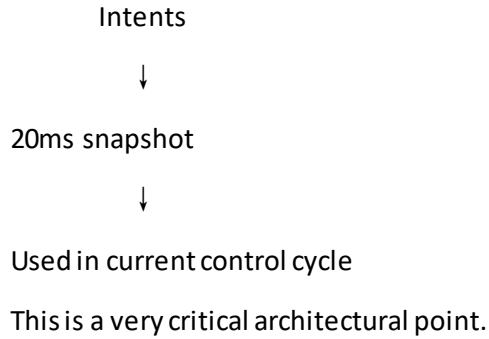
`let snapshot = intents.snapshot();`

It gets the command snapshot at this moment.

Therefore:

IPC produces command





6.1.2.2.6 Block 6: Create poweroff callback

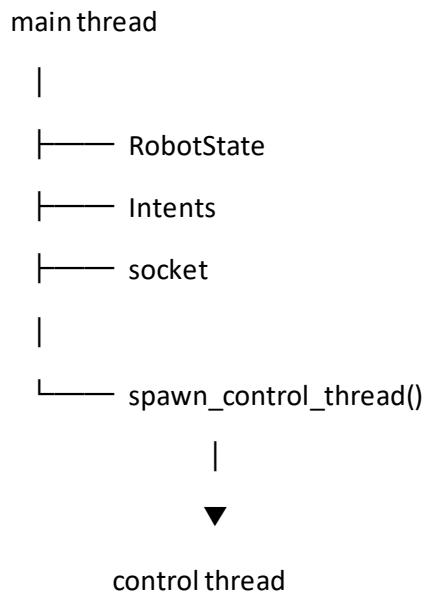
Then: `let poweroff: PowerOff = Arc::new(...)`. It does not participate in normal control. It only gives the control loop the ability to: "if shutdown is decided, truly execute `systemctl poweroff`".

So the control loop does not directly depend on shell/systemd logic, but instead receives a: `PowerOff` callback.

This also facilitates testing.

6.1.2.2.7 Block 7: Start the control thread

Then: `let control = spawn_control_thread(...)` — this is the dividing point of the entire `main()`. Up to here:

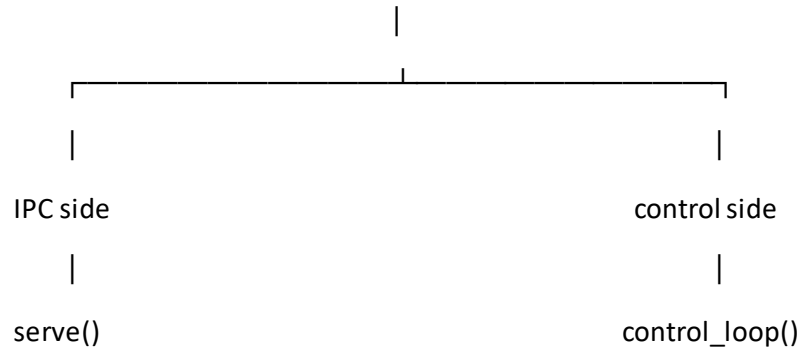


Note: The real robot control starts here and enters another execution context.

6.1.2.2.8 Block 8: Main thread starts IPC server

After the control thread starts, the main thread continues: `let serving = serve(...)`. That is to say:

robotd



Then:

```

tokio::select! {
    result = serving => ...
    _ = shutdown() => ...
}
  
```

The main thread waits here.

6.1.2.2.9 How does the IPC server itself work?

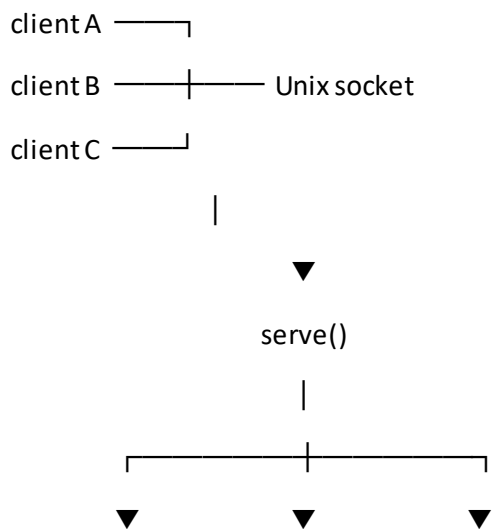
`serve()` is very simple:

loop

```

└── listener.accept()
└── Get UnixStream
└── tokio::spawn(handle(...))
  
```

That is to say:



handle handle handle

Each client can be handled independently. The code is accept → clone state/intents → tokio::spawn(handle), meaning each client is handled by its own handle.

6.1.2.2.10 What does the IPC handler do?

handle() receives JSON-RPC/NDJSON requests. It has two types of tasks:

Type 1: Queries

For example:

robot.health

robot.state

robot.policies

robot.safeToRestart

These mainly read: RobotState

Type 2: Control

For example:

robot.move

robot.head

robot.init

robot.relax

robot.do

robot.setMode

...

These do not directly control motors. Instead, requests generate intents to the control thread. Therefore, they ultimately have to wait for the next tick of the control loop.

6.1.2.2.11 When does the main thread exit?

When: SIGTERM or: IPC server error, the main flow ends. Then:

state.shutdown.store(true, ...)

Notify the control loop: "Stop.", then:

control.join()

Wait for the control thread to truly exit.

Finally:

`remove_file(socket)` removes the Unix socket.

There is also a very important detail here: it does not directly kill the control loop. The code requires:

`shutdown = true`



control loop ends current tick by itself



`Exit`



`join`

The reason is that it cannot forcefully abort in the middle of a Dynamixel bus transaction.

6.1.2.3 *Second main line: 50 Hz Control Loop*

This is the core of `robotd`. Entry point: `asyncfn control_loop<T: RobotIo>(...)`, code starts around line 1600. After it starts, it does not immediately:

`while true`

`policy`

Instead, it first establishes the control environment.

6.1.2.3.1 Control Loop initialization phase

In code order:

`control_loop()`

- └─ 1. Prepare policy configuration
- └─ 2. Clean up unloadable overrides
- └─ 3. resolve policy
- └─ 4. Create Safety
- └─ 5. Read and adopt startup pose
- └─ 6. Create Controller
- └─ 7. Create 50Hz ticker
- └─ 8. Initialize control loop state

6.1.2.3.2 Block 1: Determine current policy

6.1.2.3.3 Block 2: Create Safety

6.1.2.3.4 Block 3: Read startup pose

6.1.2.3.5 Block 4: Create Controller

6.1.2.3.6 Block 5: Establish 50 Hz ticker

Then: `let mut ticker = tokio::time::interval(period);` Normally: `period = 20ms`, then:

```
ticker.set_missed_tick_behavior(
    MissedTickBehavior::Skip
);
```

This is very important. It means:

tick 1

tick 2

tick 3 ← This one was too slow, missed

tick 4

It will not catch up:

tick 3

tick 3

Instead, it directly enters the latest time point. The control system does not want to accumulate old commands. The code also explicitly states that `Skip` is chosen to avoid backlog and time drift.

6.1.2.3.7 Enter while

Code:

```
while !state.shutdown.load(...) {
    ticker.tick().await;
```

This is the robot's true: 50 Hz main loop. Below, you must understand strictly in code order.

6.1.2.3.8 Tick Block 1: Wait for cycle

Each time: `ticker.tick().await;` waits for the next control cycle. Then: `let tick_start = Instant::now();` records this tick's start point.

6.1.2.3.9 Tick Block 2: Read sensors

6.1.2.3.10 Tick Block 3: Coast

If reading sensors fails. Code: `let sensors = coast.sample(fresh);` Meaning:

fresh read succeeded



Use fresh

fresh read failed



Temporarily use last good sample

Maximum `COAST_TICKS` = 3, which is approximately: 60 ms

6.1.2.3.11 Tick Block 4: Update IMU / odometry / fall status

6.1.2.3.12 Tick Block 5: Read Intent Snapshot

6.1.2.3.13 Tick Block 6: Safety Gate

This resolves: "Does the user want the robot to move in a way that has already timed out, or should it be restricted?"

6.1.2.3.14 Tick Block 7: Handle one-time power request

6.1.2.3.15 Tick Block 8: Handle Skill / Sit / Ground Pick

Requests such as `ground_pick`, `sit_toggle`, `skill` do not directly produce joint targets. They only change: the Controller's internal state. For example:

`robot.do kick`



`controller.start_skill()`



`active = kick`

Then the real target is only generated later by: `controller.step()`. This is very important: IPC request is "changing controller state", not "directly outputting motor targets".

6.1.2.3.16 Tick Block 9: Handle sound

6.1.2.3.17 Tick Block 10: Handle Mode Switch / Policy Reload

6.1.2.3.18 Tick Block 11: Shutdown / Battery Shutdown

6.1.2.3.19 Tick Block 12: Limp-Fall

6.1.2.3.20 Tick Block 13: Decide whether to enter driving

This is the most important condition judgment in the entire loop:

```
let driving =  
    snapshot.enabled  
    && bringup == Bringup::Ready  
    && controller.is_some()  
    && !in_limp_fall  
    && sensors.is_some()  
    && imu_warm  
    && !powered_off;
```

Only when all conditions are met does the policy truly have control of the robot.

6.1.2.3.21 Tick Block 14: Handle driving start / stop

6.1.2.3.22 Tick Block 15: Battery voltage compensation

The purpose is to maintain the effective control response of the servos as much as possible.

6.1.2.3.23 Tick Block 16: Generate Joint Targets

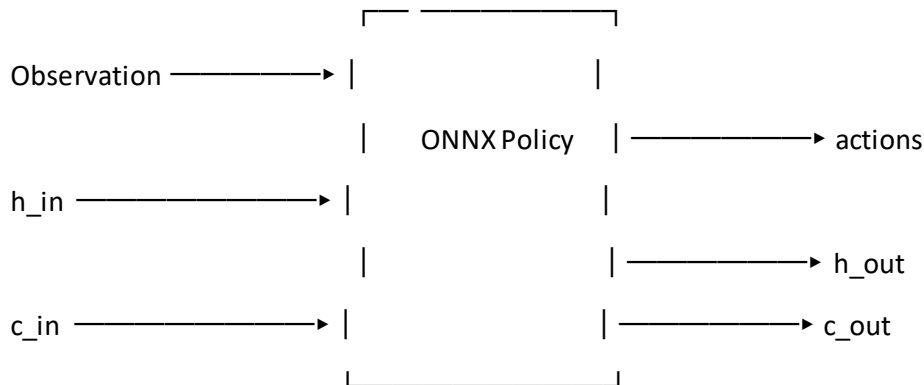
6.1.2.3.24 Tick Block 17: Controller.step()

In robotd's control loop, when the robot is truly driving, it enters:

```
controller.step(  
    sensors,  
    &command,  
    body_active,  
    period.as_secs_f64(),  
    scale_mult,  
)
```

That is to say: the control loop itself does not directly call ONNX. It passes sensors, commands, etc. to the Controller. The Controller is the control algorithm layer. The file `robotd/src/control.rs` even explicitly states at the beginning: Controller is located between `Robotlo::read` and `safety apply`, only does computation, has no IO handle itself, and therefore cannot directly control motors.

So the model is actually:



The Policy does not directly learn "absolute joint angles", but rather learns action offsets relative to the default/home pose.

6.1.2.3.24.1 *Controller Block 1: Select which Network*

6.1.2.3.24.2 *Controller Block 2: Construct Observation*

6.1.2.3.24.3 *Controller Block 3: Action → Joint Target*

6.1.2.3.24.4 *Controller Block 4: Low-pass*

6.1.2.3.24.5 *Controller Block 5: Update its own state*

6.1.2.3.25 Tick Block 18: Theremin / Chorale / Mouth

6.1.2.3.26 Tick Block 19: Finally enter Safety

Policy



targets



Expression layer modification



Safety.apply()



Robotlo

"Writing motors" is in Controller.step()



generate joint targets

← motors are not written yet here



Safety.apply()



Robotlo.write()

← This is where motors are truly written to



Dynamixel

Robotlo executes the actual Dynamixel write operation. Since these are serial motors, it actually calls Dynamixel's serial output.

6.1.2.3.27 Tick Block 20: Publish RobotState

6.1.2.3.28 Tick Block 21: Record loop health

6.1.2.4 *Some key design points of robotd*

1. After startup, it does not automatically make the robot stand up. After startup, Duck is in a limp state, preparing for update.
2. External control does not directly control motors. For example, the keyboard does not send commands directly to motors, but rather:



robot.move



That is, the keyboard sends robot movement commands via IPC (inter-process communication) as intents to the control loop (the control thread mentioned earlier), which then controls the motors. `gamepadIPCIntentscontrol loop`

-
3. Intents is the "buffer layer" between IPC and control loop
intents.rs is a very important file. It does not use mutexes ³, but rather multiple independent slots ⁴.

Every 50 Hz tick, the control loop executes:

```
let snapshot = intents.snapshot();
```

It gets this tick's input all at once. After getting the Intent:

```
let (gated, deadman) =
```

```
safety.gate(snapshot.command, snapshot.twist_age);
```

Intent



Safety.gate()



|—— Is twist timed out?

|—— Is movement allowed?

|—— What restrictions are there?

If gamepad stops sending: robot.move

Finally:

```
twist_age > deadman timeout
```

it will be cut off by the safety gate. So gamepad disconnection will not cause:

```
Last vx=1.0
```



Robot permanently maintains vx=1.0

It will enter deadman instead.

3Mutex (mutual exclusion lock) is a core mechanism for thread synchronization in multithreaded programming, whose full name is Mutual Exclusion. Its core role is to ensure that only one thread can access a shared resource at a time, completely avoiding data race problems caused by concurrent multithreaded operations.

Core principle and role

Mutual exclusion access: it is like a lock with a unique key. Before entering the critical section that accesses shared resources, a thread must acquire the lock, and release it after the operation completes; other threads attempting to acquire the already-held lock will be blocked until the lock is released.

Resolving data races: when multiple threads read and write the same shared memory (such as global counters or shared files) simultaneously, the lack of a synchronization mechanism makes results unpredictable. Mutex forces serialized access, ensuring the logical atomicity of shared data operations and avoiding data corruption.

Strict ownership: mutexes are thread-affine; only the thread holding the lock can release it. Releasing a lock from another thread directly triggers an exception or program error.

Different programming languages and systems provide mutex types adapted to different scenarios:

Type	Core features	Applicable scenarios
Plain mutex (std::mutex)	Basic and non-recursive; locking twice deadlocks	Routine protection of a single critical section; the default type in the C++ standard library

Recursive mutex (recursive_mutex)	Allows the same thread to lock multiple times; needs the same number of unlocks	Scenarios where nested calls need to repeatedly access the same shared resource
Timed mutex (timed_mutex)	Supports non-blocking locking with a timeout	Avoids threads waiting indefinitely and prevents deadlock/hangs
System-level named mutex	Visible across processes; synchronizes threads in different processes	Scenarios where multiple processes share global resources, e.g., global synchronization on Windows

Basic usage conventions:

1. The standard flow follows the "lock → operate on shared resource → unlock" pattern. Directly calling lock()/unlock() manually can easily lead to forgetting to unlock due to exceptions or early returns, causing deadlocks.

2. Modern programming practice prefers RAII-style automatic lock management classes, such as C++'s std::lock_guard or Go's defer mu.Unlock(), which lock automatically at object construction and unlock at destruction, avoiding missed unlocks at the syntax level.

3. Key precautions:

Avoid deadlocks: when multiple threads acquire multiple locks, they must apply for locks in a globally consistent fixed order; circular waiting is forbidden.

Control lock granularity: only lock the critical-section code that truly needs protection; do not perform time-consuming I/O, network requests, etc. while holding the lock, to avoid greatly reducing program concurrency performance.

Never use mutex in interrupt context: locking a mutex may cause the thread to sleep, and an interrupt context does not allow sleeping; otherwise the system will crash.

4 In the Rust official documentation and standard library implementation, the thread-related Slot is not a single standalone concept, but rather an underlying storage unit in multithreading/concurrency scenarios, distributed across three core scenarios: channel implementations, the Tokio scheduler, and thread-local storage. All definitions come from the Rust standard library source code and official public documentation.

Message Slots in Channels

This is the most clearly defined Slot concept in the Rust standard library — the core storage unit of the mpsc channel's underlying implementation, defined directly as struct Slot<T> in the standard library source code.

Core structure: each Slot contains two fields:

msg: UnsafeCell<MaybeUninit<T>>: safely stores message data in an uninitialized state, avoiding unnecessary memory initialization overhead

state: AtomicUsize: an atomic status flag using three constants to mark the Slot lifecycle: WRITE(1) means the message has been written, READ(2) means the message has been read, and DESTROY(4) means the slot is being destroyed and recycled

Block-based management of unbounded channels: unbounded channels use a linked-list block structure, where each Block<T> contains BLOCK_CAP=31 Slots, allocating multiple messages to the same block consecutively, greatly reducing memory allocation frequency and improving cache hit rate

Lock-free synchronization logic: the sending thread marks the WRITE bit via atomic operations, and the receiving thread spins waiting for that flag before reading the data — concurrent-safe message passing without any mutex

-
4. True 50 Hz tick - this is the core.
Every 20 ms: `ticker.tick().await`;
And: `MissedTickBehavior::Skip`
It is neither Burst nor Delay. Therefore, if one execution is too slow:

tick 3 ← missed

It will not:

tick 3

tick 3

tick 3

catch up.

Nor will a single delay cause the entire cycle to permanently drift. tick 1 tick 2 tick 4

6.2 ROBOTD'S ONNX MODEL DECLARATION, INSTANTIATION, AND INVOCATION

MicroDuck does not have robotd directly manage all the details of ONNX Runtime. The actual ONNX model loading, session creation, shape validation, and inference have all been

LIFO Slots in the Tokio scheduler

An optimization component specific to Tokio's multithreaded a sync scheduler, explicitly documented as a scheduling feature in the official Tokio docs, used to optimize task latency in message-passing scenarios.

Core role: each worker thread maintains a dedicated Slot. When task A wakes task B by sending a message, task B is not pushed to the tail of the task queue, but stored directly in this Slot of the current thread; at the next scheduling, task B is prioritized, greatly reducing cross-thread latency in message passing

Characteristics and configuration: tasks in this Slot cannot be stolen by other worker threads; when a task's poll execution takes a long time, overall throughput may decrease. The official `disable_lifo_slot()` configuration method can turn off this optimization, pushing all tasks into the stealable local queue

Data Slots in Thread-Local Storage (TLS)

In the thread-local storage scenario, each individual thread has a dedicated storage location allocated for TLS variables — an underlying implementation detail of the Rust standard library `std::thread::LocalKey` and the third-party `thread_local` library.

Standard library implementation: `LocalKey` dynamically allocates a dedicated Slot for each thread via the indirect-jump thunk function to store TLS data, ensuring each thread accesses a completely independent copy without any synchronization operations

Third-party library implementation: the `thread_local` library uses a hierarchical atomic pointer bucket structure to assign a unique Slot index to each thread, achieving lock-free TLS data access with performance superior to the native `pthread` TLS interface

Lifecycle rules: the first time each thread accesses a TLS variable, the corresponding Slot is lazily initialized; when the thread exits, the data in the Slot is automatically dropped via `Drop`

extracted into duck-control::policy, i.e., inside the control loop; robotd is responsible for parameter parsing, policy selection, and the 50 Hz control loop.

The most core file: duck-control/src/policy.rs.

6.2.1 Define: define an ONNX model object

File: `duck-control/src/policy.rs`. The definition stage only declares "I have an ONNX model object". The `.onnx` file has not been loaded yet, and it cannot execute yet.

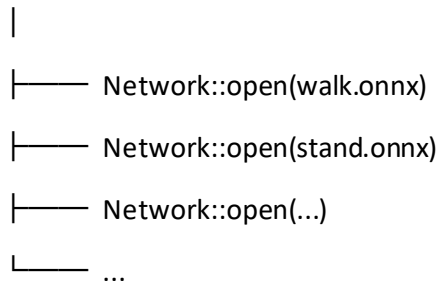
6.2.2 Create: load the `.onnx` file into a `Session`

Still in `duck-control/src/policy.rs`, creation happens inside `Network::open()`.

6.2.3 `robotd` creates it

Inside `Policy::load()`:

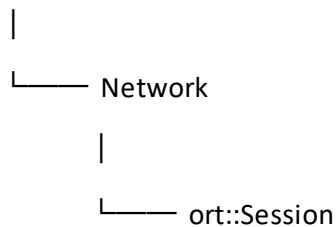
Policy::load()



robotd



Policy



For walking:

alpha_walking.onnx



Network



Session

6.2.4 Execute: call ONNX

The actual execution happens at:

```
`duck-control/src/policy.rs`
```

```
`Network::run()`.`
```

The core can be simplified to:

```
let outputs = self.session.run(inputs)?;
```

This is the most critical line.

That is:

```
```text
```

observation

|



ONNX input tensor

|



self.session.run(...)

|



ONNX Runtime

|



[1, 14]```

#### 6.2.5 How does `robotd` call it?

In the control loop, `robotd` eventually reaches:

```
self.policy.infer(&observation, net)
```

Then:

```
Controller::step()
```

|

---

▼  
Policy::infer()

|  
▼  
Network::run()

|  
▼  
session.run()

|  
▼  
ONNX

|  
▼  
14 actions

So you only need to remember this call chain:

robotd

|  
▼  
Policy::infer()

|  
▼  
Network::run()

|  
▼  
Session::run()

|  
▼  
alpha\_walking.onnx

---

### 6.2.6 Summary of the three actions

#### ① Define

Tells Rust:

```
struct Network {
 session: Session,
}
```

Meaning: a Network contains an ONNX Runtime Session.

#### ② Create

```
let session = Session::builder()
 .commit_from_file(path)?;
```

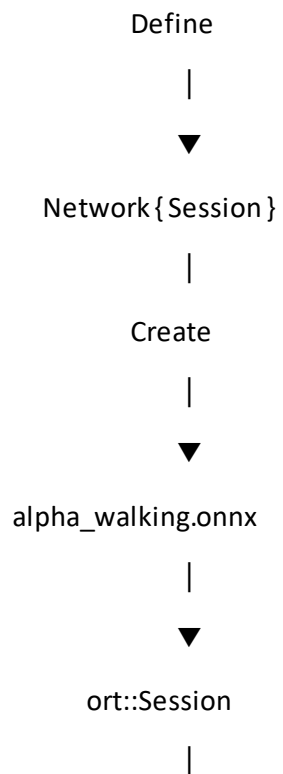
Meaning: load `alpha\_walking.onnx` into ONNX Runtime and create a `Session`.

#### ③ Execute

```
let outputs = self.session.run(inputs)?;
```

Meaning: feed the 61-dim observation to ONNX, run the neural network, and get the 14-dim action.

The simplest diagram



---

Execute

|



Session::run()

|



[1, 61] → [1, 14]

Define: declares what the object looks like.

Create = load the ONNX file to get a `Session`.\*\*

Execute = `Session::run()`, input observation, output action.\*\*

Once you clearly distinguish these three levels, the rest of `robotd`'s code will no longer confuse you.

## 6.3 FROM ONNX FILE TO PROGRAM OUTPUT: THE COMPLETE FLOW

This section talks about general ONNX concepts and workflow.

### 6.3.1 First understand: what an ONNX file itself is

An ONNX (Open Neural Network Exchange) file is essentially a "static computation graph description" — it only describes "how data flows and through which operators", and does not contain the execution engine itself. So a `.onnx` file "cannot run" by itself in a program; it must:

ONNX model file → Inference engine (Runtime) → Your program gets the output

This is exactly the reason for the layered design in the MicroDuck architecture: policy training is done in `microduck\_rl` (PyTorch + PPO) (covered in the following sections); after exporting to ONNX, the `robotd` daemon on the MicroDuck robot loads it with ONNX Runtime and runs real-time inference at 50 Hz — the training framework and the inference engine are completely decoupled. The MicroDuck control loop is the closed loop of `state → observation → neural network (ONNX) → action → physics → new state`.

### 6.3.2 The complete process (four steps)

Step 1: Confirm the "model contract" (the most easily overlooked step)

The first thing when you get an ONNX file is not to write code, but to figure out its input/output contract — shapes, data types, semantics. Taking MicroDuck's walking policy as an example:

- 
- Input: `obs`<sup>[5]</sup>, float32, shape<sup>[6]</sup>`[1, 61]`
  - Output: `actions`, float32, shape `[1, 14]` (corresponding to the actions of 14 servos)
  - Control frequency: 50 Hz (inference once every 20 ms)
  - Important detail: the observation normalizer is already baked into the ONNX graph, so callers should not normalize again.

How to inspect (choose any one):

```
```python
import onnx

model = onnx.load("policy.onnx")

print(onnx.helper.printable_graph(model.graph))
```
```

Or open it visually with the online tool [netron.app](https://netron.app).

## Step 2: Install the inference engine

The most common choice is ONNX Runtime (maintained by Microsoft, supports Python/C++/C#/Rust/Java, runs with zero dependencies on CPU; there are also prebuilt packages for ARM64 boards like RK3566/RDK X5:

```
```bash
```

5obs is short for observation. It is one of the most common variable names in reinforcement learning, robotics, and autonomous driving. Many ONNX-exported reinforcement learning policy models name their input node obs. Taking a quadruped robot as an example, it is very common in Unitree, Isaac Lab, and Isaac Gym:

```
obs =[body angular velocity 3-dim
gravity direction 3-dim
12 joint positions
12 joint velocities
previous action
target velocity]
```

6In ONNX, Tensor, and deep learning, Shape is one of the most important concepts. It can be simply understood as:

Shape describes how large each dimension of a tensor is, similar to the dimensions of a matrix. For example: input: float32[1,3,224,224], where the shape is [1,3,224,224]. `obs`^[], float32, shape^[]`[1, 61]` means: the obs variable is a 61-dimensional vector of type float32.

```
pip install onnxruntime
```

```
# If you have an NVIDIA GPU: pip install onnxruntime-gpu
```

```
...
```

Alternative engines (by scenario): TensorRT (NVIDIA acceleration), RKNN (Rockchip NPU), ONNX Runtime Web (browser — this is exactly what the official MicroDuck sandbox uses).

Step 3: Load the model and run one inference

Complete minimal Python example (corresponding to the MicroDuck contract):

```
```python
```

```
import numpy as np
```

```
import onnxruntime as ort
```

① Create the inference session (Session is a resource handle — create it once, don't put it in a loop)

```
sess = ort.InferenceSession("policy.onnx", providers=["CPUExecutionProvider"])
```

② Confirm inputs/outputs (cross-check against the model contract)

```
inp = sess.get_inputs()[0]
```

```
print(inp.name, inp.shape, inp.type) # e.g.: obs ['1', 61] tensor(float)
```

③ Prepare input: build a float32 array strictly per the contract

```
obs = np.zeros((1, 61), dtype=np.float32) # In practice, read the observation from IMU/servos
```

④ Inference: pass {input name: array}, get the output list

```
actions = sess.run(None, {inp.name: obs})[0]
```

```
print(actions.shape) # (1, 14)
```

```
...
```

C++ equivalent (the real robot's `robotd` Rust logic is essentially the same):

```
```cpp
```

```
#include <onnxruntime_cxx_api.h>
```

```
Ort::Env env(ORT_LOGGING_LEVEL_WARNING, "demo");
```

```
Ort::Session session(env, L"policy.onnx", Ort::SessionOptions{});
```

```

std::vector<int64_t> dims{1, 61};

std::vector<float> obs(61, 0.0f);

auto mem = Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault);

Ort::Value input = Ort::Value::CreateTensor<float>(mem, obs.data(), obs.size(), dims.data(),
dims.size());

auto outputs = session.Run(Ort::RunOptions{nullptr},
                           session.GetInputNames().data(), &input, 1,
                           session.GetOutputNames().data(), 1);

float* actions = outputs[0].GetTensorMutableData<float>();
...

```

In Rust you can use the `ort` crate; this is also the technical route used by the official MicroDuck stack.

Step 4: Embed into the control loop

A single `sess.run` gives you a raw tensor; in a robot program, you need to put it into a real-time loop:

```

```python
import time

RATE_HZ = 50 # Must strictly match the control frequency used
in training!

dt = 1.0 / RATE_HZ

while True:
 t0 = time.time()

 obs = read_sensors() # ① Compose the 61-dim observation
 from IMU/servo encoders

 actions = sess.run(None, {"obs": obs})[0] # ② ONNX inference → 14-dim actions
 send_to_servos(actions[0]) # ③ Send actions to the servo bus
 time.sleep(max(0, dt - (time.time() - t0))) # ④ Top up to 50 Hz

```

---

...

Engineering key points:

1. Frequency alignment: a policy trained at 50 Hz must also be called at 50 Hz during deployment, otherwise the temporal semantics of the observations are misaligned and the policy becomes unbalanced.
2. Pre/post-processing must match the model convention: for example, MicroDuck's normalization is already baked into the graph, so don't do it again; if it was not baked during export, you must do it manually in code. The official exporter `scripts/export.py` automatically bakes the normalizer — this is required for correct behavior.
3. Create the session only once, and only call `run()` in the loop; the overhead of creating a new Session each time is unacceptable in real-time control.

## 7 MICRODUCK TRAINING

---

Address: [https://github.com/pollen-robotics/microduck\\_rl](https://github.com/pollen-robotics/microduck_rl)

Getting started:

- README.md: Master the full "simulation → training → deployment" workflow (based on MuJoCo Warp simulation, PPO algorithm);
- `src/agents/`: Reinforcement learning agent implementation (reward design, policy network architecture);
- `scripts/export.py`: Script for exporting model to ONNX format (key link between simulation and hardware).

Core closed loop: Sim2Real pipeline (architecture design core)

This is the soul of Microduck's software architecture, and also the core value of open source:

1. Simulation training: Build robot model in MuJoCo Warp, train behavior policies through PPO algorithm (such as walking, fall recovery);
2. Model export: Export trained policy to ONNX format (`microduck_rl/scripts/export.py`);
3. Hardware deployment: Robot loads model via ONNX Runtime, executes in 50Hz control loop;
4. Iterative optimization: Observe differences between hardware and simulation through sensor data, adjust simulation parameters (such as friction, delay) and retrain.

---

## 8 BROWSER SIMULATION TOOL (VISUAL UNDERSTANDING OF ARCHITECTURE OPERATION)

---

No local deployment needed, experience the architecture closed loop directly in the browser:

Address: <https://huggingface.co/spaces/pollen-robotics/microduck-simulator>

Getting started:

- Operation interaction: Use WASD / gamepad to control the robot (walking, kicking, roller skating), intuitively experiencing the "input command → policy inference → motor execution" pipeline;
- Core observation: Switch between "leg / roller skating mode" (press Mkey), understand how different ONNX policies are loaded and executed;
- Debug mode: Add URL parameter ?boot=1 to enter BIOS console, view real-time logs of model loading and policy initialization (corresponding to the hardware startup process).

## 9 OFFICIAL DOCUMENTATION AND FEATURE DESCRIPTION (SUPPLEMENTARY ARCHITECTURE DESIGN PHILOSOPHY)

---

- Start from the docs/ directory in the GitHub repository, focus on agents.md (reinforcement learning policy design manual);
- Refer to the product page technical description: <https://pollen-robotics.com/microduck/> (clarifying the interaction boundary between software and hardware, such as sensor data format, motor control protocol).

## 10 RL ALGORITHMS

---